PCCST503 – MACHINE LEARNING

ASSIGNMENT 1

Design and Implementation of a Safe Semantic Planner in a Finite Cartesian State Space

Submitted by:
Shazia Habeeb

Department of Computer Science and Engineering
B.Tech Computer Science and Engineering
Academic Year: 2026

1. Introduction

Planning is an important problem in Artificial Intelligence in which an agent must determine a sequence of actions or transitions to reach a desired goal.

This assignment implements a Safe Semantic Planner for a finite Cartesian state space. The planner receives an initial state, a goal state, a set of bad states, and a collection of directed transitions. Each transition has a cost, safety value, reliability value, and availability status.

The main objective of the planner is to find a path from the initial state to the goal while ensuring that no bad state is visited. In addition to reaching the goal, the planner considers transition cost, safety distance from bad states, and reliability.

The implementation also supports dynamic changes in the planning environment. Transitions can become unavailable, new transitions can be added, and the goal state can change. The planner can then generate a revised path using the updated environment.

2. Problem Definition

The planning problem consists of a finite set of states embedded in a Cartesian space. Each state has a unique identifier and a vector representation.

The planner receives:
• Initial state
• Goal state
• Set of bad states
• Set of directed transitions

Each transition contains:
• Transition ID
• Source state
• Destination state
• Cost
• Safety score
• Reliability
• Availability flag

The planner must generate a valid sequence of transitions from the initial state to the goal state without visiting any bad state.

Optimization objectives:
1. Reach the goal state.
2. Never visit a bad state.
3. Minimize the total transition cost.
4. Maximize the minimum Euclidean distance from visited states to the nearest bad state.
5. Complete the planning process within reasonable execution time.

3. State Representation

Each state is represented using a unique 64-bit integer ID and a Cartesian embedding represented by a vector of double values.

The State structure used in the implementation is:

State {
uint64_t id;
vector embedding;
}

The ID uniquely identifies a state, while the embedding stores its position in the Cartesian space. For example, in a two-dimensional space, a state may be represented as State 0 = (0, 0), State 1 = (1, 0), and State 2 = (2, 0).

The Cartesian representation is used when calculating Euclidean distance for safety evaluation.

4. Transition Representation

A transition represents a directed connection between two states.

Each transition contains:
• id – unique transition identifier
• from – source state
• to – destination state
• cost – cost of using the transition
• safety – safety value associated with the transition
• reliability – reliability of the transition
• available – indicates whether the transition can currently be used

The transition representation allows the planner to model a dynamic directed graph.

5. Data Structures

The implementation uses several C++ data structures to efficiently represent the planning graph.

1. unordered_map – used for storing states and transitions using their unique IDs, allowing efficient average-case lookup.

2. unordered_set – used to store the IDs of bad states, allowing the planner to quickly check whether a state is unsafe.

3. vector – used for state embeddings, transition lists, bad states, state paths, and transition paths.

4. priority_queue – used as the OPEN list for the D* Lite search. Nodes with the smallest priority key are processed first.

5. g and rhs values – maintained for states as part of the D* Lite incremental search procedure.

6. Incoming and outgoing transition maps – store graph relationships and help identify neighboring states during search and replanning.

6. Planning Algorithm – D* Lite

The planner uses the D* Lite algorithm for incremental graph search.

D* Lite is suitable for this problem because the environment can change during execution. Instead of treating every update as a completely new planning problem, D* Lite maintains search information that can be updated when the environment changes.

The main search values are g(s) and rhs(s). g(s) represents the current best-known cost associated with a state. rhs(s) represents the one-step lookahead cost used to determine whether a state is locally

consistent.

The OPEN priority queue contains states that need to be processed. Each state is assigned a priority key based on its current values and heuristic estimate.

When the environment changes, such as when a transition becomes unavailable or a new transition is added, affected states can be updated and the planner can search for a revised path.

7. Heuristic Function

The Cartesian representation of states allows Euclidean distance to be used as a heuristic.

For two states s1 and s2:

$d(s1,s2) = \text{sqrt}(\Sigma(xi - yi)^2)$

For a two-dimensional state:

$d = \text{sqrt}((x1-x2)^2 + (y1-y2)^2)$

The heuristic estimates the remaining spatial distance between the current state and the goal state. Because the state space is represented using Cartesian coordinates, Euclidean distance provides a natural estimate of spatial distance to the goal.

8. Safety Computation

Safety is evaluated using the distance between a visited state and the nearest bad state.

For every state in the path, the planner calculates its Euclidean distance to each bad state and selects the minimum distance.

Distance(s) = min d(s,b), where b belongs to the set of bad states.

The minimum safety distance of the complete path is then calculated as the minimum distance among all visited states. A larger minimum distance indicates that the path maintains a larger safety margin from bad states.

Bad states themselves are completely prohibited. If a transition leads to a bad state, that transition is treated as unavailable by the planner.

9. Cost and Reliability

Each transition has an associated cost and reliability.

The total path cost is calculated by adding the costs of all transitions in the selected path:

Total Cost = c1 + c2 + ... + cn

Reliability is also accumulated across the selected transitions. The planner uses reliability as one of the factors considered during path selection.

Therefore, the planner does not simply search for any path. It evaluates paths using multiple properties such as cost, safety, and reliability.

10. Dynamic Environment and Replanning

The planning environment is dynamic. Therefore, the planner must be able to handle changes after an initial path has already been generated.

The implementation demonstrates three types of changes:

1. Transition Failure – a transition that was previously available can become unavailable. The planner then searches for an alternative path.

2. Goal Update – the goal state can change during execution. The planner can generate a new path from the initial state to the updated goal.

3. Transition Addition – a new transition can be inserted into the graph. If the new transition provides a better route, the planner can discover the improved path.

D* Lite is suitable for dynamic environments because it is designed for incremental replanning. Search information can be updated rather than treating every environmental change as an entirely unrelated planning problem.

11. Experimental Test Cases

Test Case	Purpose	Expected Result
1	Basic Reachability	Find S → A → B → G
2	Bad State Avoidance	Avoid bad state X
3	Safety Margin	Select safer valid route
4	Dynamic Transition	Find alternative after failure
5	Goal Update	Find route to new goal
6	Transition Addition	Discover new shortcut

12. Experimental Results

Test Case	Result	Total Cost	Min. Safety Distance	Explored States	Bad States
1. Basic Reachability	Success	3.000	N/A	4	0
2. Bad State Avoidance	Success	3.000	1.414	5	0
3. Safety Margin	Success	9.000	1.118	5	0
4. Dynamic Transition – Before	Success	2.000	N/A	4	0
4. Dynamic Transition – After	Success	6.000	N/A	4	0
5. Goal Update – Original	Success	2.000	N/A	3	0
5. Goal Update – New	Success	4.000	N/A	3	0
6. Transition Addition – Before	Success	6.000	N/A	4	0
6. Transition Addition – After	Success	1.000	N/A	2	0

13. Result Analysis

All six test cases successfully produced valid paths.

In Test Case 1, the planner successfully found the unique path from the initial state to the goal.

In Test Case 2, the planner avoided the bad state and selected the alternative safe path. The number of

bad states visited was zero.

In Test Case 3, the planner selected a path with a higher total cost while maintaining a safety distance from the bad states. This demonstrates the trade-off between cost and safety.

In Test Case 4, when a transition became unavailable, the original path could no longer be used. The planner successfully generated an alternative path.

In Test Case 5, changing the goal resulted in a new valid path to the updated goal.

In Test Case 6, adding a shortcut transition reduced the total path cost from 6.000 to 1.000. The planner successfully discovered the newly added transition.

Most importantly, zero bad states were visited in all test cases.

14. Time Complexity

Let V be the number of states and E be the number of available transitions.

The planner uses graph search with a priority queue. For a typical priority-queue-based graph search, the search complexity is approximately:

O((V + E) log V)

Safety computation requires checking the distance to the bad states. If there are B bad states and the path/search examines V states, the worst-case safety calculation can require **O(VB)**.

15. Space Complexity

The graph requires approximately **O(V + E)** space. Additional storage is required for g-values, rhs-values, state information, transition information, and the OPEN priority queue.

16. Conclusion

A Safe Semantic Planner was successfully designed and implemented for a finite Cartesian state space.

The planner finds paths while avoiding bad states and considers transition cost, safety distance, reliability, and availability. All six test cases successfully produced valid paths, with zero bad states visited. The implementation also demonstrated adaptation to transition failures, goal changes, and newly added transitions.

17. Software and Repository

Implementation language: C++
Main source file: main.cpp
Repository: safe-semantic-planner
GitHub: https://github.com/shaziahabeeb/safe-semantic-planner